

PyQt5 Signals, Slots & Events

Triggering actions in response to user behaviors and GUI events

by Martin Fitzpatrick 📅 Last updated Dec 15 [PyQt5](#) 📁 Getting started with PyQt5

🚩 PyQt5 Tutorial — Getting started with PyQt5

- ☐ Creating your first app with PyQt5
- ☒ 🚩 PyQt5 Signals, Slots & Events
- ☐ PyQt5 Widgets
- ☐ PyQt5 Layouts
- ☐ PyQt5 Toolbars & Menus — QAction
- ☐ PyQt5 Dialogs and Alerts
- ☐ Creating Additional Windows in PyQt5

This tutorial is also available for [PySide6](#), [PyQt6](#) and [PySide2](#)

So far we've created a window and added a simple *push button* widget to it, but the button doesn't do anything. That's not very useful at all -- when you create GUI applications you typically want them to do something! What we need is a way to connect the action of *pressing the button* to making something happen. In Qt, this is provided by *signals* and *slots* or *events*.

Table of Contents

- Signals & Slots
 - QPushButton Signals
 - Receiving data
 - Storing data
 - Changing the interface
 - Connecting widgets together directly
- Events
 - Mouse events
 - Context menus
 - Event hierarchy

Signals & Slots

Signals are notifications emitted by widgets when *something* happens. That something can be any number of things, from pressing a button, to the text of an input box changing, to the text of the window changing. Many signals are initiated by user action, but this is not a rule.

In addition to notifying about something happening, signals can also send data to provide additional context about what happened.



You can also create your own custom signals, which we'll explore later.

Slots is the name Qt uses for the receivers of signals. In Python any function (or method) in your application can be used as a slot -- simply by connecting the signal to it. If the signal sends data, then the receiving function will receive that data too. Many Qt widgets also have their own built-in slots, meaning you can hook Qt widgets together directly.

Let's take a look at the basics of Qt signals and how you can use them to hook widgets up to make things happen in your apps.

Save the following app outline to a file named `app.py`.



PyQt/PySide 1:1 Coaching with Martin Fitzpatrick — Save yourself time and frustration. Get one on one help with your Python GUI projects. Working together with you I'll identify issues and suggest fixes, from bugs and usability to architecture and maintainability.

Book Now

60 mins (\$195)

PYTHON

```
import sys
from PyQt5.QtWidgets import QApplication, QMainWindow, QPushButton

class MainWindow(QMainWindow):

    def __init__(self):
        super().__init__()

        self.setWindowTitle("My App")

app = QApplication(sys.argv)

window = MainWindow()
window.show()

app.exec()
```

QPushButton Signals

Our simple application currently has a `QMainWindow` with a `QPushButton` set as the central widget. Let's start by hooking up this button to a custom Python method. Here we create a simple custom slot named `the_button_was_clicked` which accepts the `clicked` signal from the `QPushButton`.

PYTHON

```
import sys
from PyQt5.QtWidgets import QApplication, QMainWindow, QPushButton

class MainWindow(QMainWindow):
    def __init__(self):
        super().__init__()
```



```
self.setWindowTitle("My App")

button = QPushButton("Press Me!")
button.setCheckable(True)
button.clicked.connect(self.the_button_was_clicked)

# Set the central widget of the window.
self.setCentralWidget(button)

def the_button_was_clicked(self):
    print("Clicked!")

app = QApplication(sys.argv)

window = MainWindow()
window.show()

app.exec()
```



Run it! If you click the button you'll see the text "Clicked!" on the console.

PYTHON

```
Clicked!
Clicked!
Clicked!
Clicked!
```

Receiving data

That's a good start! We've heard already that signals can also send *data* to provide more information about what has just happened. The `.clicked` signal is no exception, also providing a *checked* (or toggled) state for the button. For normal buttons this is always `False`, so our first slot ignored this data. However, we can make our button *checkable* and see the effect.

In the following example, we add a second slot which outputs the *checkstate*.

PYTHON

```
class MainWindow(QMainWindow):
    def __init__(self):
        super().__init__()
```



```
self.setWindowTitle("My App")

button = QPushButton("Press Me!")
button.setCheckable(True)
button.clicked.connect(self.the_button_was_clicked)
button.clicked.connect(self.the_button_was_toggled)

self.setCentralWidget(button)

def the_button_was_clicked(self):
    print("Clicked!")

def the_button_was_toggled(self, checked):
    print("Checked?", checked)
```



Run it! If you press the button you'll see it highlighted as *checked*. Press it again to release it. Look for the *check state* in the console.

PYTHON

```
Clicked!
Checked? True
Clicked!
Checked? False
Clicked!
Checked? True
Clicked!
Checked? False
Clicked!
Checked? True
```

You can connect as many slots to a signal as you like and can respond to different versions of signals at the same time on your slots.

Storing data

Often it is useful to store the current *state* of a widget in a Python variable. This allows you to work with the values like any other Python variable and without accessing the original widget. You can either store these values as individual variables or use a dictionary if you prefer. In the next example we store the *checked* value of our button in a variable called

`button_is_checked` on `self`.

```

class MainWindow(QMainWindow):
    def __init__(self):
        super().__init__()

        self.button_is_checked = True

        self.setWindowTitle("My App")

        button = QPushButton("Press Me!")
        button.setCheckable(True)
        button.clicked.connect(self.the_button_was_toggled)
        button.setChecked(self.button_is_checked)

        self.setCentralWidget(button)

    def the_button_was_toggled(self, checked):
        self.button_is_checked = checked

        print(self.button_is_checked)

```

First we set the default value for our variable (to `True`), then use the default value to set the initial state of the widget. When the widget state changes, we receive the signal and update the variable to match.

You can use this same pattern with any PyQt widgets. If a widget does not provide a signal that sends the current state, you will need to retrieve the value from the widget directly in your handler. For example, here we're checking the checked state in a *pressed* handler.

```

class MainWindow(QMainWindow):
    def __init__(self):
        super().__init__()

        self.button_is_checked = True

        self.setWindowTitle("My App")

        self.button = QPushButton("Press Me!")
        self.button.setCheckable(True)
        self.button.released.connect(self.the_button_was_released)
        self.button.setChecked(self.button_is_checked)

        self.setCentralWidget(self.button)

    def the_button_was_released(self):
        self.button_is_checked = self.button.isChecked()

```



```
print(self.button_is_checked)
```



We need to keep a reference to the button on `self` so we can access it in our slot.

The *released* signal fires when the button is released, but does not send the check state, so instead we use `.isChecked()` to get the check state from the button in our handler.

Changing the interface

So far we've seen how to accept signals and print output to the console. But how about making something happen in the interface when we click the button? Let's update our slot method to modify the button, changing the text and disabling the button so it is no longer clickable. We'll also turn off the *checkable* state for now.

PYTHON

```
class MainWindow(QMainWindow):
    def __init__(self):
        super().__init__()

        self.setWindowTitle("My App")

        self.button = QPushButton("Press Me!")
        self.button.clicked.connect(self.the_button_was_clicked)

        self.setCentralWidget(self.button)

    def the_button_was_clicked(self):
        self.button.setText("You already clicked me.")
        self.button.setEnabled(False)

        # Also change the window title.
        self.setWindowTitle("My Oneshot App")
```

Again, because we need to be able to access the `button` in our `the_button_was_clicked` method, we keep a reference to it on `self`. The text of the button is changed by passing a `str` to `.setText()`. To disable a button call `.setEnabled()` with `False`.



Run it! If you click the button the text will change and the button will become unclickable.

You're not restricted to changing the button that triggers the signal, you can do *anything you want* in your slot methods. For example, try adding the following line to

`the_button_was_clicked` method to also change the window title.

PYTHON

```
self.setWindowTitle("A new window title")
```

Most widgets have their own signals -- and the `QMainWindow` we're using for our window is no exception. In the following more complex example, we connect the

`.windowTitleChanged` signal on the `QMainWindow` to a custom slot method.

In the following example we connect the `.windowTitleChanged` signal on the `QMainWindow` to a method slot `the_window_title_changed`. This slot also receives the new window title.

PYTHON

```
from PyQt5.QtWidgets import QApplication, QMainWindow, QPushButton

import sys
from random import choice

window_titles = [
    'My App',
    'My App',
    'Still My App',
    'Still My App',
    'What on earth',
    'What on earth',
    'This is surprising',
    'This is surprising',
    'Something went wrong'
]

class MainWindow(QMainWindow):
    def __init__(self):
        super().__init__()

        self.n_times_clicked = 0

        self.setWindowTitle("My App")

        self.button = QPushButton("Press Me!")
```




```
self.button.clicked.connect(self.the_button_was_clicked)

self.windowTitleChanged.connect(self.the_window_title_changed)

# Set the central widget of the Window.
self.setCentralWidget(self.button)

def the_button_was_clicked(self):
    print("Clicked.")
    new_window_title = choice(window_titles)
    print("Setting title: %s" % new_window_title)
    self.setWindowTitle(new_window_title)

def the_window_title_changed(self, window_title):
    print("Window title changed: %s" % window_title)

    if window_title == 'Something went wrong':
        self.button.setDisabled(True)

app = QApplication(sys.argv)

window = MainWindow()
window.show()

app.exec()
```

First we set up a list of window titles -- we'll select one at random from this list using Python's built-in `random.choice()`. We hook our custom slot method `the_window_title_changed` to the window's `.windowTitleChanged` signal.

When we click the button the window title will change at random. If the new window title equals "Something went wrong" the button will be disabled.



Run it! Click the button repeatedly until the title changes to "Something went wrong" and the button will become disabled.

There are a few things to notice in this example.

Firstly, the `windowTitleChanged` signal is not *a/ways* emitted when setting the window title. The signal only fires if the new title is *changed* from the previous one. If you set the same title multiple times, the signal will only be fired the first time. It is important to double-check the conditions under which signals fire, to avoid being surprised when using them in your app.

Secondly, notice how we are able to *chain* things together using signals. One thing happening -- a button press -- can trigger multiple other things to happen in turn. These subsequent effects do not need to know *what* caused them, but simply follow as a consequence of simple rules. This *decoupling* of effects from their triggers is one of the key concepts to understand when building GUI applications. We'll keep coming back to this throughout the book!

In this section we've covered signals and slots. We've demonstrated some simple signals and how to use them to pass data and state around your application. Next we'll look at the widgets which Qt provides for use in your applications -- together with the signals they provide.

Connecting widgets together directly

So far we've seen examples of connecting widget signals to Python methods. When a signal is fired from the widget, our Python method is called and receives the data from the signal. But you don't *always* need to use a Python function to handle signals -- you can also connect Qt widgets directly to one another.

In the following example, we add a `QLineEdit` widget and a `QLabel` to the window. In the `__init__` for the window we connect our line edit `.textChanged` signal to the `.setText` method on the `QLabel`. Now any time the text changes in the `QLineEdit` the `QLabel` will receive that text to its `.setText` method.

PYTHON

```
from PyQt5.QtWidgets import QApplication, QMainWindow, QLabel, QLineEdit, QVBoxLayout
import sys

class MainWindow(QMainWindow):
    def __init__(self):
        super().__init__()

        self.setWindowTitle("My App")

        self.label = QLabel()

        self.input = QLineEdit()
        self.input.textChanged.connect(self.label.setText)

        layout = QVBoxLayout()
        layout.addWidget(self.input)
        layout.addWidget(self.label)
```

```
container = QWidget()
container.setLayout(layout)

# Set the central widget of the Window.
self.setCentralWidget(container)
```

```
app = QApplication(sys.argv)
```

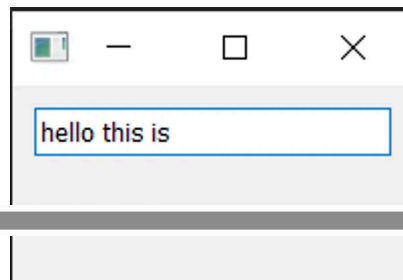
```
window = MainWindow()
window.show()
```

```
app.exec()
```

Notice that in order to connect the input to the label, the input and label must both be defined. This code adds the two widgets to a layout, and sets that on the window. We'll cover layouts in detail later, you can ignore it for now.



Run it! Type some text in the upper box, and you'll see it appear immediately on the label.



Any text typed in the input immediately appears on the label.

Most Qt widgets have *slots* available, to which you can connect any signal that emits the same *type* that it accepts. The widget documentation has the slots for each widget listed under "Public Slots". For example, see [QLabel](#) .

Events

Every interaction the user has with a Qt application is an *event*. There are many types of event, each representing a different type of interaction. Qt represents these events using



event objects which package up information about what happened. These events are passed to specific *event handlers* on the widget where the interaction occurred.

Get your project to market with focused expert help

Take your app to market!

We help startups and research groups take their applications from *prototype scripts* to *fully-functional* applications ready to release. With years of *practical experience* shipping for Windows, macOS and Linux.



Ready to launch?

Find out More

Launch Support for your Python Applications

Comprehensive code review · **Bugfixes** & improvements · **Maintainability** advice and **architecture** improvements · **Design and usability** assessment · Suggestions and tips to **expand your knowledge** · **Packaging and distribution** help for Windows, Mac & Linux · [Find out more.](#)

By defining custom, or extended *event handlers* you can alter the way your widgets respond to these events. Event handlers are defined just like any other method, but the name is specific for the type of event they handle.

One of the main events which widgets receive is the `QMouseEvent`. `QMouseEvent` events are created for each and every mouse movement and button click on a widget. The



following event handlers are available for handling mouse events --

Event handler	Event type moved
<code>mouseMoveEvent</code>	Mouse moved
<code>mousePressEvent</code>	Mouse button pressed
<code>mouseReleaseEvent</code>	Mouse button released
<code>mouseDoubleClickEvent</code>	Double click detected

For example, clicking on a widget will cause a `QMouseEvent` to be sent to the `.mousePressEvent` event handler on that widget. This handler can use the event object to find out information about what happened, such as what triggered the event and where specifically it occurred.

You can intercept events by sub-classing and overriding the handler method on the class. You can choose to filter, modify, or ignore events, passing them up to the normal handler for the event by calling the parent class function with `super()`. These could be added to your main window class as follows. In each case `e` will receive the incoming event.

PYTHON

```
import sys

from PyQt5.QtCore import Qt
from PyQt5.QtWidgets import QApplication, QLabel, QMainWindow, QTextEdit

class MainWindow(QMainWindow):
    def __init__(self):
        super().__init__()
        self.label = QLabel("Click in this window")
        self.setCentralWidget(self.label)

    def mouseMoveEvent(self, e):
        self.label.setText("mouseMoveEvent")

    def mousePressEvent(self, e):
        self.label.setText("mousePressEvent")

    def mouseReleaseEvent(self, e):
        self.label.setText("mouseReleaseEvent")

    def mouseDoubleClickEvent(self, e):
        self.label.setText("mouseDoubleClickEvent")

app = QApplication(sys.argv)
```



```
window = MainWindow()
window.show()

app.exec()
```



Run it! Try moving and clicking (and double-clicking) in the window and watch the events appear.

You'll notice that mouse move events are only registered when you have the button pressed down. You can change this behavior by calling `setMouseTracking(True)` on the window and the label. In this case, the `__init__()` method could look something like this:

PYTHON

```
class MainWindow(QMainWindow):
    def __init__(self):
        super().__init__()
        self.setMouseTracking(True)
        self.label = QLabel("Click in this window")
        self.label.setMouseTracking(True)
        self.setCentralWidget(self.label)
```



We need to use `.setMouseTracking` on *both* the label and window here, because the label completely covers the window and would otherwise block the events.

You may also notice that the press (click) and double-click events both fire when the button is pressed down. Only the release event fires when the button is released. Typically, to register a click from a user, you should watch for both the mouse down *and* the release.

Inside the event handlers you have access to an event object. This object contains information about the event and can be used to respond differently depending on what exactly has occurred. We'll look at the mouse event objects next.

Mouse events

All mouse events in Qt are tracked with the `QMouseEvent` object, with information about the event being readable from the following event methods.



Method	Returns
<code>.button()</code>	Specific button that triggered this event
<code>.buttons()</code>	State of all mouse buttons (OR'ed flags)
<code>.globalPos()</code>	Application-global position as a <code>QPoint</code>
<code>.globalX()</code>	Application-global <i>horizontal</i> X position
<code>.globalY()</code>	Application-global <i>vertical</i> Y position
<code>.pos()</code>	Widget-relative position as a <code>QPoint</code> <i>integer</i>
<code>.posF()</code>	Widget-relative position as a <code>QPointF</code> <i>float</i>

You can use these methods within an event handler to respond to different events differently, or ignore them completely. The positional methods provide both *global* and *local* (widget-relative) position information as `QPoint` objects, while buttons are reported using the mouse button types from the `Qt` namespace.

For example, the following allows us to respond differently to a left, right or middle click on the window.

PYTHON

```
def mousePressEvent(self, e):
    if e.button() == Qt.LeftButton:
        # handle the left-button press in here
        self.label.setText("mousePressEvent LEFT")

    elif e.button() == Qt.MiddleButton:
        # handle the middle-button press in here.
        self.label.setText("mousePressEvent MIDDLE")

    elif e.button() == Qt.RightButton:
        # handle the right-button press in here.
        self.label.setText("mousePressEvent RIGHT")

def mouseReleaseEvent(self, e):
    if e.button() == Qt.LeftButton:
        self.label.setText("mouseReleaseEvent LEFT")

    elif e.button() == Qt.MiddleButton:
        self.label.setText("mouseReleaseEvent MIDDLE")

    elif e.button() == Qt.RightButton:
        self.label.setText("mouseReleaseEvent RIGHT")

def mouseDoubleClickEvent(self, e):
    if e.button() == Qt.LeftButton:
        self.label.setText("mouseDoubleClickEvent LEFT")
```

```
elif e.button() == Qt.MidddleButton:
    self.label.setText("mouseDoubleClickEvent MIDDLE")

elif e.button() == Qt.RightButton:
    self.label.setText("mouseDoubleClickEvent RIGHT")
```

The button identifiers are defined in the Qt namespace, as follows --

Identifier	Value (binary)	Represents
Qt.NoButton	0 (000)	No button pressed, or the event is not related to button press.
Qt.LeftButton	1 (001)	The left button is pressed
Qt.RightButton	2 (010)	The right button is pressed.
Qt.MidddleButton	4 (100)	The middle button is pressed.

On left-handed mice the left and right button positions are reversed, i.e. pressing the right-most button will return `Qt.LeftButton`. This means you don't need to account for the mouse orientation in your code.

Context menus

Context menus are small context-sensitive menus which typically appear when right clicking on a window. Qt has support for generating these menus, and widgets have a specific event used to trigger them. In the following example we're going to intercept the `.contextMenuEvent` a `QMainWindow`. This event is fired whenever a context menu is *about to be* shown, and is passed a single value `event` of type `QContextMenuEvent`.

To intercept the event, we simply override the object method with our new method of the same name. So in this case we can create a method on our `MainWindow` subclass with the name `contextMenuEvent` and it will receive all events of this type.

PYTHON

```
import sys

from PyQt5.QtCore import Qt
from PyQt5.QtWidgets import QAction, QApplication, QLabel, QMainWindow, QMenu
```



```

class MainWindow(QMainWindow):
    def __init__(self):
        super().__init__()

    def contextMenuEvent(self, e):
        context = QMenu(self)
        context.addAction(QAction("test 1", self))
        context.addAction(QAction("test 2", self))
        context.addAction(QAction("test 3", self))
        context.exec(e.globalPos())

app = QApplication(sys.argv)

window = MainWindow()
window.show()

app.exec()

```

If you run the above code and right-click within the window, you'll see a context menu appear. You can set up `.triggered` slots on your menu actions as normal (and re-use actions defined for menus and toolbars).

When passing the initial position to the `exec` function, this must be relative to the parent passed in while defining. In this case we pass `self` as the parent, so we can use the global position.

Just for completeness, there is actually a signal-based approach to creating context menus.

PYTHON

```

class MainWindow(QMainWindow):
    def __init__(self):
        super().__init__()
        self.show()

        self.setContextMenuPolicy(Qt.CustomContextMenu)
        self.customContextMenuRequested.connect(self.on_context_menu)

    def on_context_menu(self, pos):
        context = QMenu(self)
        context.addAction(QAction("test 1", self))
        context.addAction(QAction("test 2", self))
        context.addAction(QAction("test 3", self))
        context.exec(self.mapToGlobal(pos))

```

Event hierarchy

In PyQt every widget is part of two distinct hierarchies: the Python object hierarchy, and the Qt layout hierarchy. How you respond or ignore events can affect how your UI behaves.

Python inheritance forwarding

Often you may want to intercept an event, do something with it, yet still trigger the default event handling behavior. If your object is inherited from a standard widget, it will likely have sensible behavior implemented by default. You can trigger this by calling up to the parent implementation using `super()`.



This is the Python parent class, not the PyQt `.parent()`.

```
def mousePressEvent(self, event):  
    print("Mouse pressed!")  
    super().mousePressEvent(event)
```

PYTHON

The event will continue to behave as normal, yet you've added some non-interfering behavior.

Layout forwarding

When you add a widget to your application, it also gets another *parent* from the layout. The parent of a widget can be found by calling `.parent()`. Sometimes you specify these parents manually, such as for `QMenu` or `QDialog`, often it is automatic. When you add a widget to your main window for example, the main window will become the widget's parent.

When events are created for user interaction with the UI, these events are passed to the *uppermost* widget in the UI. So, if you have a button on a window, and click the button, the button will receive the event first.



If the first widget cannot handle the event, or chooses not to, the event will *bubble up* to the parent widget, which will be given a turn. This *bubbling* continues all the way up nested widgets, until the event is handled or it reaches the main window.

In your own event handlers you can choose to mark an event as *handled* calling

```
.accept() --
```

PYTHON

```
class CustomButton(QPushButton)
    def mousePressEvent(self, e):
        e.accept()
```

Alternatively, you can mark it as *unhandled* by calling `.ignore()` on the event object. In this case the event will continue to bubble up the hierarchy.

PYTHON

```
class CustomButton(QPushButton)
    def event(self, e):
        e.ignore()
```

If you want your widget to appear transparent to events, you can safely ignore events which you've actually responded to in some way. Similarly, you can choose to accept events you are not responding to in order to silence them.

Create GUI Applications with Python & Qt5 by Martin Fitzpatrick — (PyQt5 Edition)

The hands-on guide to making apps with Python — Over 15,000 copies sold!

More info

Get the book

Mark As Complete

Continue with [PyQt5 Tutorial](#)

Return to [Create Desktop GUI Applications with PyQt5](#)

PyQt5 Signals, Slots & Events was written by [Martin Fitzpatrick](#).



Martin Fitzpatrick has been developing Python/Qt apps for 8 years. Building desktop applications to make data-analysis tools more user-friendly, Python was the obvious choice. Starting with Tk, later moving to wxWidgets and finally adopting PyQt. Martin founded PythonGUIs to provide easy to follow GUI programming tutorials to the Python community. He has written a number of popular [Python books](#) on the subject.

PyQt5 Signals, Slots & Events was published in [tutorials](#) on May 21, 2019 (updated December 15, 2025) . Feedback & Corrections [can be submitted here](#).

[QT](#)[PYQT](#)[PYQT5](#)[FOUNDATION](#)[PYTHON](#)[QT5](#)[PYQT5-FOUNDATION](#)[Where do I begin?](#)[Learn the fundamentals](#)[Databases & SQL](#)[Data Science](#)[Packaging & Distribution](#)[QML/QtQuick](#)[Raspberry Pi](#)[Games](#)[Intermediate Tutorials](#)

Sections

[Installation](#)[First steps with PySide6](#)[First steps with PyQt6](#)[Example Python Apps](#)[Widget Library](#)[Simple PyQt6 & PySide6 documentation](#)[Reusable code & snippets](#)[Frequently Asked Questions](#)



Tutorials

[Which Python GUI library?](#)

[PyQt5 tutorial](#)

[PyQt6 tutorial](#)

[PySide2 tutorial](#)

[PySide6 tutorial](#)

[Tkinter tutorial](#)

[Latest articles](#)

Books & Downloads

📄 Your Downloads

[PyQt5 Book / PySide2 Book](#)

[PyQt6 Book / PySide6 Book](#)

[Python Packaging Book](#)

[📄 Book Source Code](#)

[▶️ PyQt6 Video Course](#)

Community & Consulting

💬 Python GUIS Forum

[! Feedback & Corrections](#)

[Consulting](#)

[Mentoring](#)

[Contact me](#)

[Licensing, Privacy & Legal](#)



Python GUIs Copyright ©2014-2026 Martin Fitzpatrick

Tutorials CC-BY-NC-SA Sitemap Changelog Powered by Python Public code BSD & MIT